

LION FIT PRO

Architecture Decision Records (ADR)

1. Introducción

Un Architecture Decision Record (ADR) es un documento que captura una decisión arquitectónica importante tomada en el contexto de un proyecto de software. Cada ADR describe el contexto del problema, la decisión tomada, la justificación de esa decisión, las alternativas que se consideraron y las consecuencias (positivas y negativas) de la elección.

Este documento compila 8 ADRs del proyecto Lion Fit Pro, cubriendo las decisiones clave de backend, base de datos, autenticación, frontend móvil, comunicación HTTP, despliegue y patrones de seguridad.

Formato de cada ADR:

Campo	Descripción
Contexto	El problema o necesidad que origina la decisión
Decisión	Que se decidió hacer concretamente
Justificación	Por que esta decisión y no otra
Consecuencias	Efectos positivos y negativos de la decisión
Alternativas	Opciones que se evaluaron y descartaron
Referencia de código	Archivos concretos del repositorio donde se aplica la decisión

2. Resumen de ADRs

ID	Título	Fecha	Estado	Tecnología clave
ADR-001	Adopción de NestJS como framework backend	Noviembre 2025	Aceptada	NestJS 11
ADR-002	Uso de Prisma ORM con PostgreSQL	Noviembre 2025	Aceptada	Prisma ORM 6
ADR-003	Autenticación basada en JWT con Passport.js	Noviembre 2025	Aceptada	JWT + Passport
ADR-004	Vue.js 3 + Ionic Framework para la aplicación móvil	Noviembre 2025	Aceptada	Vue.js 3 + Ionic 8
ADR-005	Axios como cliente HTTP con interceptores centralizados	Noviembre 2025	Aceptada	Axios Interceptors

ADR-006	Despliegue en AWS EC2 con servidor Node.js integrado	Noviembre 2025	Aceptada	AWS EC2
ADR-007	Estructura de rutas REST con prefijo global api/v1	Noviembre 2025	Aceptada	REST API v1
ADR-008	Autenticación en el frontend con guards de Vue Router	Noviembre 2025	Aceptada	Vue Router Guards

3. ADRs Detallados

ADR-001 Adopción de NestJS como framework backend

ID	Título	Fecha	Estado	Decisores
ADR-001	Adopción de NestJS como framework backend	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El proyecto requería un backend robusto con soporte nativo para TypeScript, inyección de dependencias, modularidad y facilidad para implementar autenticación JWT y validaciones. Se evaluaron Express.js puro, Fastify y NestJS.

Decisión

Se adopta NestJS 11 como framework principal del backend, aprovechando su arquitectura modular (AppModule, AuthModule, ExerciseModule, WorkoutModule, ExerciseRecordModule), el sistema de guards para protección de rutas y la integración con Passport.js para JWT.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
• Arquitectura modular que mapea directamente el dominio del negocio (un módulo por entidad).	✓ Estructura consistente y predecible entre todos los módulos del backend.	◆ Express.js: Mas ligero pero requiere configuración manual de todo el ecosistema.
• Inyección de dependencias nativas que facilita el testing y la separación de responsabilidades.	✓ Validación automática de DTOs con class-validator y ValidationPipe global.	◆ Fastify: Mayor rendimiento pero menor ecosistema de plugins para autenticación.
• Integración lista para usar con class-validator, Passport, JWT y Prisma.	✓ JwtAuthGuard reutilizable con decorador @Public() para rutas abiertas.	◆ Hono: Moderno y rápido pero sin la madurez de NestJS para proyectos empresariales.
• TypeScript de primera clase, consistente con el frontend en Vue.js + TypeScript.	✗ Curva de aprendizaje inicial para desarrolladores sin experiencia con el patron de decoradores.	

Decoradores declarativos (@Controller, @Injectable, @Get, @Post) que reducen el boilerplate.	✗ Mayor overhead que Express puro para APIs muy simples.	
--	--	--

ADR-002 Uso de Prisma ORM con PostgreSQL

ID	Título	Fecha	Estado	Decisores
ADR-002	Uso de Prisma ORM con PostgreSQL	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El sistema necesita persistir entidades relacionadas: User, Routine, Exercise, Muscle, RoutineExercise, Workout y ExerciseRecord. Se requería un ORM con migraciones automáticas, tipado fuerte y compatibilidad con NestJS.

Decisión

Se utiliza Prisma ORM 6 con PostgreSQL. Cada servicio de dominio extiende PrismaClient directamente (ExerciseService, ExerciseRecordService) e implementa OnModuleInit para gestionar la conexión al inicio del módulo.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
Schema declarativo en prisma/schema.prisma que sirve como fuente única de verdad del modelo de datos.	✓ Sin inconsistencias entre el modelo de datos y los tipos TypeScript del backend.	◆ TypeORM: Mayor integración con NestJS pero decoradores más verbosos y migraciones menos intuitivas.
Generación automática de tipos TypeScript desde el schema, eliminando discrepancias entre BD y código.	✓ Migraciones reproducibles: migration.sql versionadas en el repositorio.	◆ Sequelize: Maduro pero sin generación automática de tipos TypeScript.
Sistema de migraciones versionadas (4 migraciones identificadas en el proyecto).	✓ deleteMany() para eliminar registros por workoutId en una sola operación atómica.	◆ Knex (query builder): Más control pero sin ORM ni tipos automáticos.
Queries type-safe: this.exercise.findMany(), this.exerciseRecord.deleteMany() con autocompletado.	✗ Patron 'extends PrismaClient' por servicio crea múltiples instancias de conexión.	

• Soporte nativo para relaciones (User->Workout->ExerciseRecord, Exercise->Muscle).	✗ Recomendación oficial de Prisma es usar un PrismaService singleton inyectable.	
--	--	--

ADR-003 Autenticación basada en JWT con Passport.js

ID	Título	Fecha	Estado	Decisores
ADR-003	Autenticación basada en JWT con Passport.js	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

La aplicación móvil necesita autenticar usuarios y proteger rutas. Al ser una SPA móvil (Ionic/Capacitor) sin sesiones del servidor, se requería un mecanismo stateless. Se evaluaron JWT, sesiones con cookies y OAuth2.

Decisión

Se implementa autenticación JWT con @nestjs/jwt y passport-jwt. El token se firma con un secreto configurable vía JWT SECRET (variable de entorno). El frontend almacena el token en localStorage y lo envía en el header Authorization: Bearer. El JwtAuthGuard protege todas las rutas excepto las marcadas con @Public().

Justificación (Rationale)	Consecuencias	Alternativas consideradas
• JWT es stateless: compatible con despliegue en AWS EC2 sin necesidad de session store.	✓ Arquitectura stateless lista para escalar horizontalmente.	◆ Sesiones con cookies: Estado en servidor, incompatible con arquitectura stateless en AWS.
• El decorador @Public() permite marcar endpoints abiertos (login, register) sin duplicar lógica.	✓ JwtAuthGuard reutilizable: una sola configuración protege todos los módulos.	◆ OAuth2/Keycloak: Más seguro pero añade complejidad operacional innecesaria para MVP.
• El interceptor de Axios en api.service.ts inyecta automáticamente el token en cada request.	✓ Refresco automático del header Authorization via interceptor de Axios.	◆ Supabase Auth: SaaS que simplifica la autenticación pero genera dependencia de terceros.
• bcrypt para hash de contraseñas con salt rounds (seguridad ante ataques de rainbow table).	✗ JWT_SECRET con fallback hardcodeado ('your-secret-key') en	

	jwt.strategy.ts es un riesgo de seguridad en producción.	
• TokenPayload tipado con { email, sub, name } para evitar accesos arbitrarios al payload.	✗ Sin mecanismo de revocación de tokens (logout no invalida el JWT en el servidor).	
	✗ Token almacenado en localStorage es vulnerable a ataques XSS.	

ADR-004 Vue.js 3 + Ionic Framework para la aplicación móvil

ID	Título	Fecha	Estado	Decisores
ADR-004	Vue.js 3 + Ionic Framework para la aplicación móvil	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El proyecto requería una aplicación móvil multiplataforma (Android prioritario) con componentes UI nativos, navegación por tabs y rendimiento aceptable. Se evaluaron React Native, Flutter, Ionic + React e Ionic + Vue.

Decisión

Se utiliza Vue.js 3 con Composition API e Ionic 8 como framework de componentes UI móviles, compilado para Android via Capacitor 7. El enrutamiento se maneja con @ionic/vue-router.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
• Ionic provee componentes nativos (IonPage, IonHeader, IonSelect, IonCard) sin necesidad de implementar UI desde cero.	✓ Un solo codebase para web y Android (multiplataforma).	◆ React Native: Mayor rendimiento nativo pero requiere conocimiento de React.
• Capacitor 7 permite compilar la web app como APK Android con acceso a APIs nativas.	✓ Componentes Ionic manejan estilos nativos según la plataforma automáticamente.	◆ Flutter: Mejor rendimiento pero lenguaje Dart ajeno al equipo.
• Vue 3 Composition API con <script setup> reduce el boilerplate respecto a Options API.	✓ Ecosistema completo: @ionic/vue-router, IonTabs, IonModal incluidos.	◆ Ionic + React: Similar pero el equipo tenía más experiencia con Vue.

• TypeScript integrado en el proyecto (tsconfig.json, vue-tsc en el build).	✗ Rendimiento inferior a React Native o Flutter para animaciones complejas.	
• Vite como bundler para Hot Module Replacement rápido en desarrollo.	✗ El TrainingPage.vue usa datos simulados (registroHoy = null hardcodeado) sin integración real con el backend.	

ADR-005 Axios como cliente HTTP con interceptores centralizados

ID	Título	Fecha	Estado	Decisores
ADR-005	Axios como cliente HTTP con interceptores centralizados	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

La aplicación móvil necesita comunicarse con el backend REST. Se requería un cliente HTTP que soporte interceptores para inyección automática del token JWT, manejo centralizado de errores 401 y tipado de respuestas.

Decisión

Se utiliza Axios con una instancia centralizada en api.service.ts. Se configuran dos interceptores: uno de request que adjunta el Bearer token desde localStorage, y uno de response que maneja el error 401 redirigiendo al login y registra en consola todos los errores de API.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
• Un solo punto de configuración (baseUrl: http://18.217.60.222:3000/api/v1) para todos los servicios.	✓ DRY principal: la lógica de autenticación se escribe una vez en el interceptor.	◆ Fetch API nativa: Sin dependencias externas pero sin interceptores nativos.
• Interceptor de request elimina la necesidad de pasar el token manualmente en cada llamada.	✓ Fácil de extender: agregar headers globales o logging en un solo lugar.	◆ ofetch (unjs): Más moderno pero menor adopción en el ecosistema Vue/Ionic.
• Manejo centralizado del error 401 (token expirado o invalido) con limpieza de localStorage.	✗ La URL base esta hardcodeada con IP publica (18.217.60.222) en lugar de variable de entorno.	◆ Apollo Client: Aplica solo si se hubiera usado GraphQL en lugar de REST.

• Todos los servicios (exerciseService, userService, workoutService) importan la misma instancia 'api'.	✗ El interceptor de response tiene console.log en respuestas exitosas (ruido en producción).	
	✗ useRouter() dentro del interceptor de error es un antipatrón fuera del contexto de Vue.	

ADR-006 Despliegue en AWS EC2 con servidor Node.js integrado

ID	Título	Fecha	Estado	Decisores
ADR-006	Despliegue en AWS EC2 con servidor Node.js integrado	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El backend necesitaba un servidor accesible desde internet para que la aplicación Android pueda conectarse. Se evaluaron AWS EC2, Railway, Render y Heroku como plataformas de despliegue.

Decisión

El backend NestJS se despliega en una instancia AWS EC2 en la IP 18.217.60.222, puerto 3000. El NestExpressApplication sirve adicionalmente los archivos estáticos del frontend compilado desde el directorio frontend/dist, consolidando frontend y backend en un único proceso Node.js.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
• Control total sobre la instancia: configuración de puertos, variables de entorno y recursos.	✓ Arquitectura simplificada para MVP: un servidor, una IP, un proceso.	◆ Railway/Render: Despliegue automático desde Git pero menos control y posibles costos.
• Servir el frontend desde NestJS elimina la necesidad de un servidor web separado (Nginx, Apache).	✓ Sin costos adicionales de CDN o servidor web separado.	◆ AWS ECS + RDS: Arquitectura más robusta pero complejidad excesiva para un MVP.
• Un único proceso simplifica el despliegue para un proyecto académico/MVP.	✗ IP pública hardcodeada en el frontend (18.217.60.222) no es una buena práctica.	◆ Vercel (frontend) + Railway (backend): Separación de responsabilidades pero

		configuración CORS adicional.
• CORS configurado con origin: '*' permite conexiones desde la app Android y el navegador.	✗ CORS con origin '*' es inseguro para la producción real.	
• Puerto configurable vía variable de entorno process.env.PORT con fallback a 3000.	✗ Sin HTTPS: la comunicación entre la app Android y el servidor no está cifrada.	
	✗ Sin balanceador de carga ni alta disponibilidad.	

ADR-007 Estructura de rutas REST con prefijo global api/v1

ID	Título	Fecha	Estado	Decisores
ADR-007	Estructura de rutas REST con prefijo global api/v1	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

El backend expone múltiples recursos (user, auth, exercise, workout, routine, routine-exercise, exercise-record). Se requería una convención de rutas consistente que soporte versionado futuro y sea legible para el frontend.

Decisión

Se aplica un prefijo global 'api/v1' via app.setGlobalPrefix('api/v1') en main.ts. Los controladores usan rutas de recurso en plural o dominio: /auth, /user, /exercise, /workout, /routine-exercise, /exercise-record. Las rutas anidadas usan parámetros: /exercise-record/workout/:workoutId.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
• El prefijo v1 permite introducir v2 en el futuro sin romper clientes existentes.	✓ Versionado de API preparado para evolucionar sin romper la app móvil.	◆ Versionado por header (Accept: application/vnd.api+json; version=1): Más REST puro pero complejo.
• Rutas RESTful predecibles: GET /exercise (listar), POST /exercise (crear), DELETE /exercise/:id (eliminar).	✓ Rutas autodescriptivas que facilitan el consumo desde el frontend.	◆ Rutas sin versión (/api/exercise): Simple pero sin ruta de migración para versiones futuras.

El frontend Axios tiene una sola baseURL que incluye el prefijo api/v1.	✗ La ruta /workout/by-date-range podría confundirse con un recurso :id en Express si no se define antes.	◆ GraphQL: Un solo endpoint pero cambio de paradigma completo.
Ruta especializada /workout/by-date-range con query params (userId, startDate, endDate) para consultas de historial.	✗ userId se envía como query param en lugar de extraerse del JWT (implicación de seguridad).	

ADR-008 Autenticación en el frontend con guards de Vue Router

ID	Título	Fecha	Estado	Decisores
ADR-008	Autenticación en el frontend con guards de Vue Router	Noviembre 2025	Aceptada	Equipo de desarrollo

Contexto

La aplicación Vue necesitaba proteger las rutas autenticadas (/tabs/*sin duplicar lógica de autenticación en cada componente. Se evaluaron guards de navegación globales, guards por ruta y composables de autenticación.

Decisión

Se implementa la función authGuard en router/index.ts que verifica la existencia del token 'authToken' en localStorage. La función se aplica como beforeEnter en cada ruta protegida individualmente: training, daily-log, exercise/:workoutId y weekly-history.

Justificación (Rationale)	Consecuencias	Alternativas consideradas
Guard como función pura reusable sin acoplamiento a ningún componente Vue.	✓ Lógica de protección centralizada en un único lugar del router.	◆ Guard global beforeEach: Mas DRY pero requiere lista de rutas públicas explícita.
beforeEnter por ruta (no global) da control explícito sobre cuales rutas se protegen.	✓ Fácil de extender: agregar roles o permisos modifica solo authGuard.	◆ Pinia store de autenticación: Estado reactivo pero mayor complejidad para MVP.
Verificación simple por existencia del token (sin	✗ El guard solo verifica existencia del token, no su validez o expiración.	◆ Composable useAuth(): Más reusable pero

validar expiración) suficiente para MVP.		requiere refactorización de los componentes.
• La capa de verificación real de expiración ocurre en el backend vía JwtAuthGuard.	✗ Si el token expira, el usuario solo se enteraría cuando el backend retorna 401.	
	✗ 'authToken' como string mágico en vez de constante exportada.	
